

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: «Алгоритм Ахо-Корасик»

Студентка гр. 4345

Бодарева С.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать алгоритм Ахо-Корасик.

Задание.

Задание 1

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$)

Вторая – число n ($1 \leq n \leq 3000$), каждая из следующих n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел – i p

Где i – позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номер шаблона.

Вход:

NTAG

3

TAGT

TAG

T

Выход:

2 2

2 3

Задание 2

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Вход:

Текст ($T, 1 \leq |T| \leq 100000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Sample Input:

ACTANCA

A\$\$\$A\$

\$

Sample Output:

1

Вариант 1. На месте джокера может быть любой символ, за исключением заданного.

Основные теоретические положения.

Алгоритм Ахо-Корасик – алгоритм поиска подстроки, реализует поиск множества подстрок из словаря в данной строке.

Префиксное дерево или бор — это структура данных для компактного хранения строк. Она устроена в виде дерева, где на рёбрах между вершинами написаны символы, а некоторые вершины помечены терминальными. Суффиксная ссылка — это ссылка на узел, соответствующий самому длинному суффиксу, который не заводит бор в тупик

Выполнение.

Для реализации алгоритма Ахо-Корасик было создано 2 класса:

- *Класс Node* – класс для хранения узла бора. Методы класса:
 1. *def __init__(self, name)*. Инициализатор объекта класса. Объект класса содержит следующие поля:
 - *name* – содержит имя узла, т.е. все символы от корня до текущего узла;
 - *children* – словарь детей узла;
 - *is_terminal* – информация о том, терминальная вершина или нет;
 - *suff_link, exit_link* – суффиксная и входная ссылки узла;
 - *pattern_nums, pattern_len* – хранят порядковый номер шаблона и его длину.
- *Класс Trie* – класс для хранения и обработки бора. Методы класса:
 1. *def __init__(self)* – инициализирует объект класса *Node* с именем «» – корень бора;
 2. *def add_word(self, word: str, word_num: int) -> None* – функция для добавления шаблона в бор.

На вход поступает слово и его порядковый номер в списке. Обходим дерево, начиная с корня. Рассматриваем текущий символ слова, если у текущего узла есть переход по этому символу, то совершаем его, если перехода нет, то создаем

дочерний узел и совершаем переход. После добавления последнего символа слова в бор, соответствующая ему вершина объявляется терминальной, в список *pattern_nums* добавляется порядковый номер шаблона, в поле *pattern_len* его длина.

3. *def create_links(self) -> None* – функция создания суффиксных и конечных ссылок для всех узлов бора.

С помощью очереди обходим узлы бора в ширину, начиная с корня. Для каждого узла по очереди рассматриваем все его дочерние узлы.

Для каждого дочернего узла *child*, достигнутого при переходе по *char*, ищем суффиксную ссылку. Суффиксная ссылка должна указывать на узел, соответствующий самому длинному собственному суффиксу строки *child.name*, присутствующей в боре. Для этого надо подниматься по суффиксным ссылкам вверх, пока не найдется узел с переходом по символу *char* или не встретится корень.

Затем для каждого узла вычисляется выходная ссылка. Выходная ссылка нужна, чтобы при поиске не пропустить вхождение более коротких шаблонов, являющихся суффиксами текущей строки. Если узел, на который указывает суффиксная ссылка *child* является терминальным, то выходная ссылка ведёт в него, иначе перенимается его выходная ссылка.

4. *def aho_corasick(self, text: str) -> list* – функция, реализующая алгоритм Ахо-Корасик.

Обходим текст посимвольно, а текущий узел бора, начиная с корня, храним в *current*. Для каждого символа пытаемся совершить переход из текущего узла. Если перехода по текущему символу нет и текущий узел – не корень, то поднимаемся вверх по суффиксным ссылкам, пока не найдем

узел с таким переходом или не окажемся в корне. Если переход по текущему символу найден, то совершаем его, иначе – остаемся в корне.

После перехода проверяем найденные вхождения: обходим все выходные ссылки, начиная с *current*. Если узел терминальный – вычисляем позицию начала вхождения и добавляем в результирующий массив позицию начала вхождения и номер шаблона. Обход выходных ссылок позволяет не пропускать более короткие шаблоны, являющиеся суффиксами строки, соответствующей текущему узлу.

Метод возвращает отсортированный массив пар позиций вхождения шаблонов и их порядковых номеров.

Для второго задания алгоритм был изменен. В классическом алгоритме в бор добавляются цельные шаблоны с порядковыми номерами, в задаче с джокером шаблон разбивается на части по символу-джокеру, и в бор добавляются эти части, а вместо порядкового номера сохраняется смещение от начала шаблона. Также есть отличия в самом поиске. В первом задании при нахождении вхождения в результирующий массив записывается пара (позиция, номер вхождения), а в задаче с джокером ведется массив *match_count*, где для каждой позиции текста считается, сколько частей шаблона туда попало. Когда находится вхождение очередной части, через её длину и смещение относительно начала шаблона вычисляется позиция, где должен начинаться весь шаблон, счётчик для этой позиции увеличивается. В конце позиция считается верной только если счётчик равен общему числу частей шаблона – то есть нашлись все части шаблона на соответствующих местах. В соответствии с заданием варианта 1, результат дополнительно проверяется. Для каждой позиции шаблона, где стоит джокер, проверяется, что соответствующий символ не является запрещенным. Позиции, содержащие запрещенный символ убираются из ответа.

Оценим временную сложность реализованного алгоритма:

1. Сложность построения бора – метод *add_word*: $O(m)$, где m – сумма длин всех шаблонов
2. Сложность построения суффиксных и выходных ссылок – метод *create_links*: при построении суффиксной ссылки для узла на глубине d происходит подъем вверх, родитель уже имеет суффиксную ссылку, следовательно каждый узел участвует в подъеме не более одного раза за весь обход. В задании лабораторной работы мощность алфавита равна 5, поэтому итоговая сложность: $O(m)$
3. Сложность поиска – метод *aho_corasick*: основной цикл: $O(t)$, где t – длина текста. Каждый проход по цепочке *exit_link* дает ровно одно вхождение в результат, следовательно суммарное число шагов по *exit_link* равно количеству вхождений *res*. Итоговая сложность $O(t + res)$
4. Сложность стандартной сортировки *result.sort()*: $O(res \cdot \log(res))$
Итоговая сложность $O(m + t + res \cdot \log(res))$

Пространственная сложность: $O(m + t + res)$

Исходный код программы см. в Приложении А.

Результаты тестирования см. в таблице 1.

Таблица 1. Результаты тестирования

№	Входные данные	Выходные данные	Комментарии
1	NTAG 3 TAGT TAG T	2 2 2 3	Пример из задания.
2	ACTANGA A\$\$A\$ \$	1	Пример из задания.
3	xabvccbababcaх ab??c? ? v	8	Вхождение шаблона с позиции 2 удалено из ответа, поскольку вместо джокера на 4 месте текста расположен запрещающий символ v.

Вывод

Был изучен и реализован алгоритм Ахо-Корасик, решающий задачу точного поиска набора образцов в тексте. Кроме того, исходная программа была модифицирована для задачи точного поиска для одного образца с джокером. В соответствии с заданием варианта 1 программа принимает на вход ещё и запрещающий символ, который не может быть на месте джокера.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ.

Название файла: step1.py

```
from collections import deque

class Node:
    def __init__(self):
        self.children = {}
        self.is_terminal = False
        self.suff_link = None
        self.exit_link = None
        self.pattern_nums = []
        self.pattern_len = 0

class Trie:
    def __init__(self):
        self.root = Node()

    def add_word(self, word: str, word_num: int) -> None:
        current = self.root
        for char in word:
            if char not in current.children:
                current.children[char] = Node()
            current = current.children[char]
        current.is_terminal = True
        current.pattern_len = len(word)
        current.pattern_nums.append(word_num)

    def create_links(self) -> None:
        queue = deque([self.root])
        while queue:
            current = queue.popleft()
            for char, child in current.children.items():
                link = current.suff_link
                while link is not None and char not in link.children:
                    link = link.suff_link
                child.suff_link = link.children[char] if link else self.root
                if child.suff_link is child:
                    child.suff_link = self.root

                if child.suff_link.is_terminal:
                    child.exit_link = child.suff_link
                else:
                    child.exit_link = child.suff_link.exit_link
            queue.append(child)

    def aho_corasick(self, text: str) -> list:
        result = []
        current = self.root
        for i in range(len(text)):
            char = text[i]
            while current is not self.root and char not in current.children:
                current = current.suff_link
            if char in current.children:
                current = current.children[char]
            node = current
```

```

        while node is not self.root:
            if node.is_terminal:
                pos = i - node.pattern_len + 2
                for num in node.pattern_nums:
                    result.append((pos, num))
            node = node.exit_link
            if node is None:
                break
    result.sort()
    return result

trie = Trie()
text = input()
n = int(input())
for i in range(n):
    trie.add_word(input(), i+1)
trie.create_links()
res = trie.aho_corasick(text)
for i in res:
    print(i[0], i[1])

```

Название файла: step2.py

```

from collections import deque

class Node:
    def __init__(self):
        self.children = {}
        self.is_terminal = False
        self.suff_link = None
        self.exit_link = None
        self.pattern_starts = []
        self.pattern_len = 0

class Trie:
    def __init__(self):
        self.root = Node()

    def add_word(self, word: str, word_start: int) -> None:
        current = self.root
        for char in word:
            if char not in current.children:
                current.children[char] = Node()
            current = current.children[char]
        current.is_terminal = True
        current.pattern_len = len(word)
        current.pattern_starts.append(word_start)

    def create_links(self) -> None:
        queue = deque([self.root])
        while queue:
            current = queue.popleft()
            for char, child in current.children.items():
                link = current.suff_link
                while link is not None and char not in link.children:
                    link = link.suff_link
                child.suff_link = link.children[char] if link else self.root
                if child.suff_link is child:
                    child.suff_link = self.root

```

```

        if child.suff_link.is_terminal:
            child.exit_link = child.suff_link
        else:
            child.exit_link = child.suff_link.exit_link
        queue.append(child)

    def aho_corasick(self, text: str, pattern_len: int, count_word: int) ->
list:
    match_count = [0] * (len(text)+1)
    current = self.root
    for i in range(len(text)):
        char = text[i]
        while current is not self.root and char not in current.children:
            current = current.suff_link
        if char in current.children:
            current = current.children[char]
        node = current
        while node is not self.root:
            if node.is_terminal:
                for pos in node.pattern_starts:
                    start = i - node.pattern_len - pos + 2
                    if 1 <= start <= len(text) - pattern_len + 1:
                        match_count[start] += 1
            node = node.exit_link
        if node is None:
            break
    result = []
    for i in range(1, len(text) + 1):
        if match_count[i] == count_word:
            result.append(i)
    return result

trie = Trie()
text = input()
pattern = input()
wildcard = input()
parts = pattern.split(wildcard)
pos = 0
count_word = 0
starts = []
for i in range(len(parts)):
    part = parts[i]
    if part:
        trie.add_word(part, pos)
        starts.append(pos)
        count_word += 1
    pos += len(part) + 1
trie.create_links()
result = trie.aho_corasick(text, len(pattern), count_word)
for i in result:
    print(i)

```